

MPC Is All You Need

Jeremy Morgan
August 6 2025

Project goal



Few shot manipulation task execution with no offline training

- One or few demonstrations
- No offline training phase

Section 1: Model Predictive Control Recap

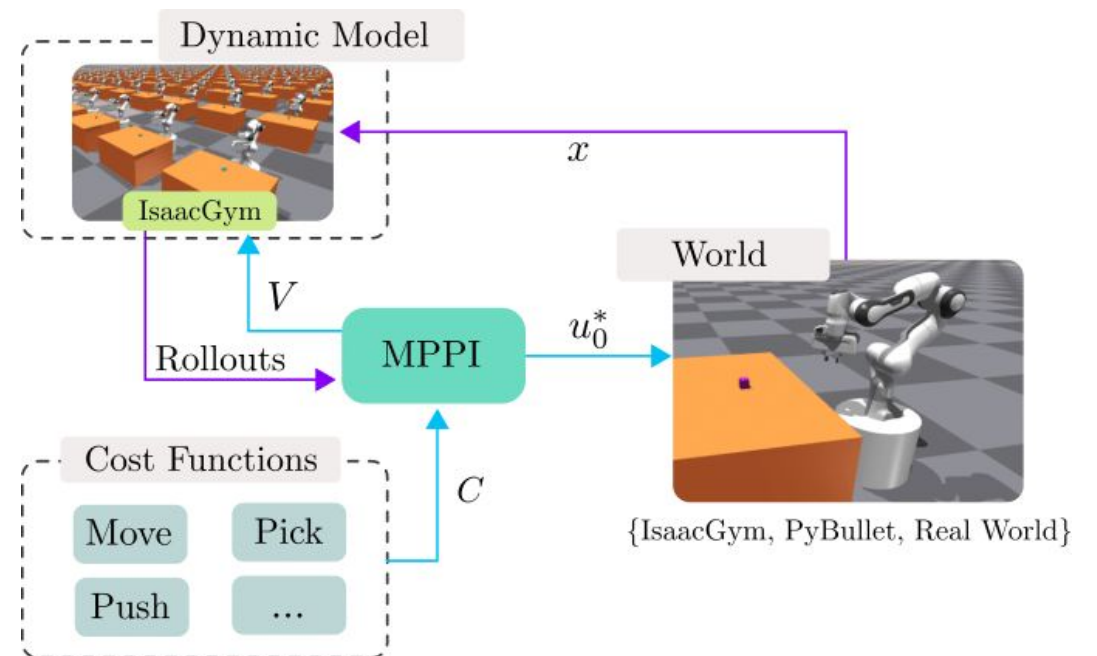
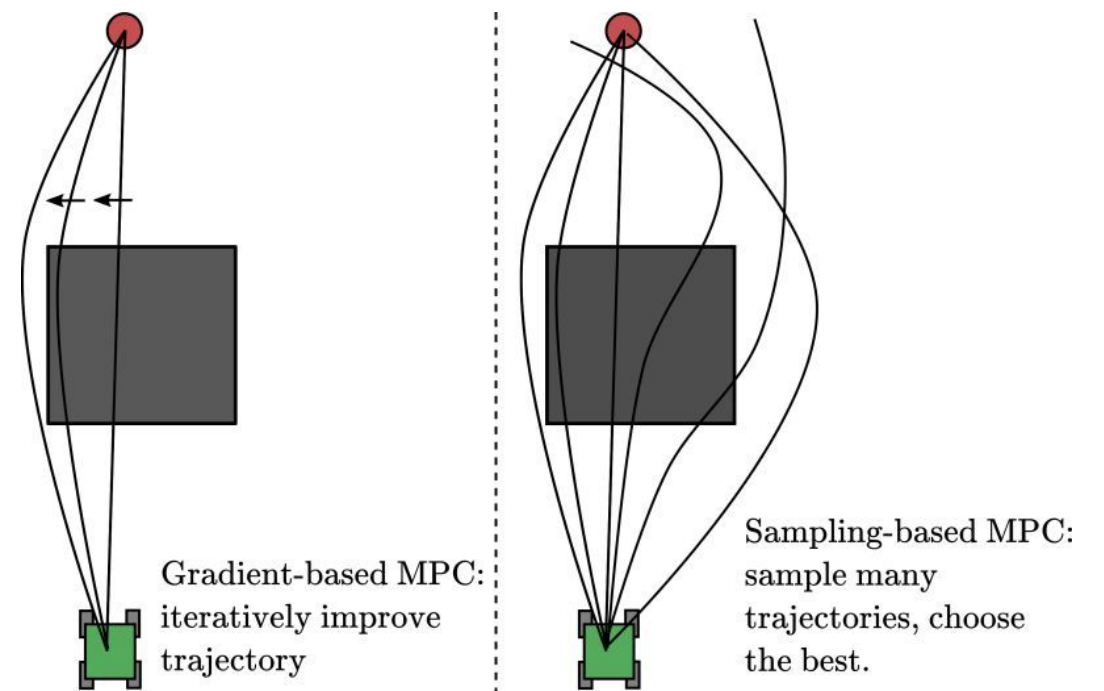
MPC Intro

Components

1. Dynamics Model
2. Action Sampler
3. Reward Function

For this project

1. Dynamics Model: Sim digital twin, generated via a Real2Sim method
2. Action Sampler: MPPI, or predictive sampling
3. Reward Function: TBD



Optimal Control Taxonomy



Optimal control

🌐 19 languages ▾

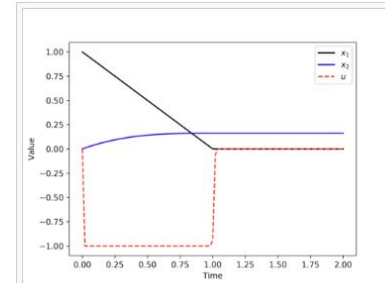
[Article](#) [Talk](#)

[Read](#) [Edit](#) [View history](#) [Tools](#) ▾

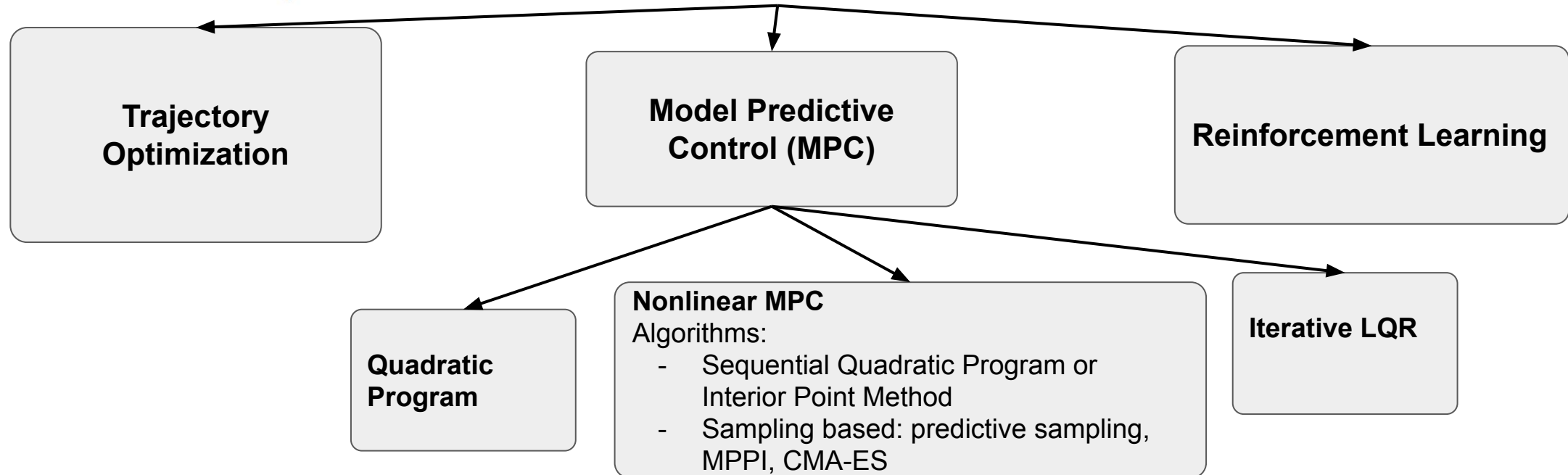
From Wikipedia, the free encyclopedia

Optimal control theory is a branch of [control theory](#) that deals with finding a [control](#) for a [dynamical system](#) over a period of time such that an [objective function](#) is optimized.^[1] It has numerous applications in science, engineering and operations research. For example, the dynamical system might be a [spacecraft](#) with controls corresponding to rocket thrusters, and the objective might be to reach the [Moon](#) with minimum fuel expenditure.^[2] Or the dynamical system could be a nation's [economy](#), with the objective to minimize [unemployment](#); the controls in this case could be [fiscal](#) and [monetary policy](#).^[3] A dynamical system may also be introduced to embed [operations research problems](#) within the framework of optimal control theory.^{[4][5]}

Optimal control is an extension of the [calculus of variations](#), and is a mathematical



Optimal control problem benchmark (Luus) with an integral objective, inequality, and differential constraint



Section 2: Reward Functions

Reward Function Desiderata



1. Easy to generate: no programming or manual knob-tuning
 - a. If from demonstrations: Fewer needed than LfD would require
2. Generalizes well to different environments
3. Reliably solves the task when minimized



Reward Function Landscape

Common ones

1. Hand coded
2. Visual Reward Function
3. VLM-based

...

Others

- Preference learning

Reward Function #1: Hand coded

Manually specified. Typically tuned via graduate student descent

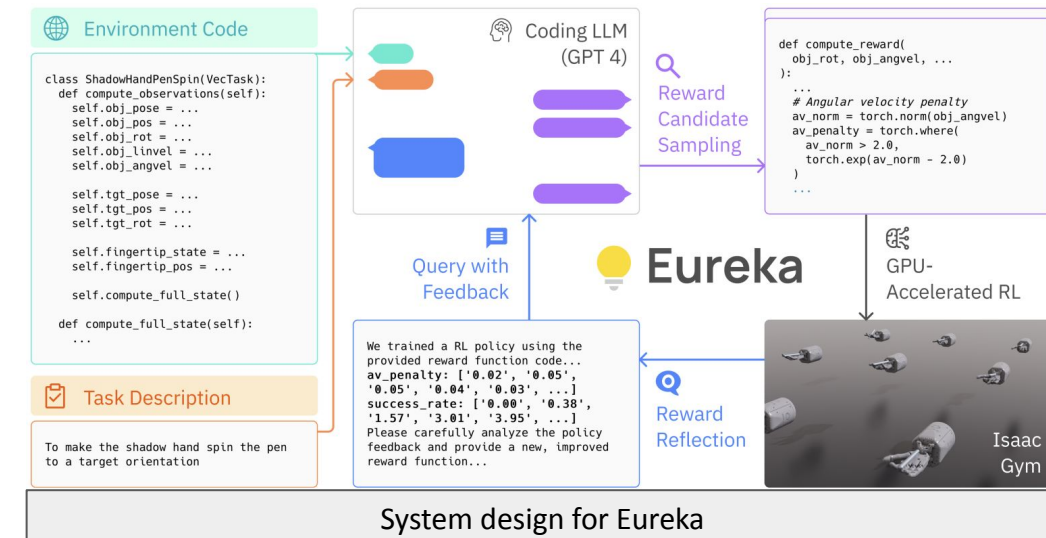
- Attempts at automation by work like Eureka^[0]
 - LLM writes code directly, observes policy improvement
- RL for learning the weights of an existing reward expression (the Q, c in a QP)

```
def compute_dense_reward(self, obs: Any, action: torch.Tensor, info: Dict):
    tcp_to_obj_dist = torch.linalg.norm(
        self.cube.pose.p - self.agent.tcp_pose.p, axis=1
    )
    reaching_reward = 1 - torch.tanh(5 * tcp_to_obj_dist)
    reward = reaching_reward

    is_grasped = info["is_grasped"]
    reward += is_grasped

    obj_to_goal_dist = torch.linalg.norm(
        self.goal_site.pose.p - self.cube.pose.p, axis=1
    )
    place_reward = 1 - torch.tanh(5 * obj_to_goal_dist)
    reward += place_reward * is_grasped
```

Reward function snippet from ManiSkill for PickCube task

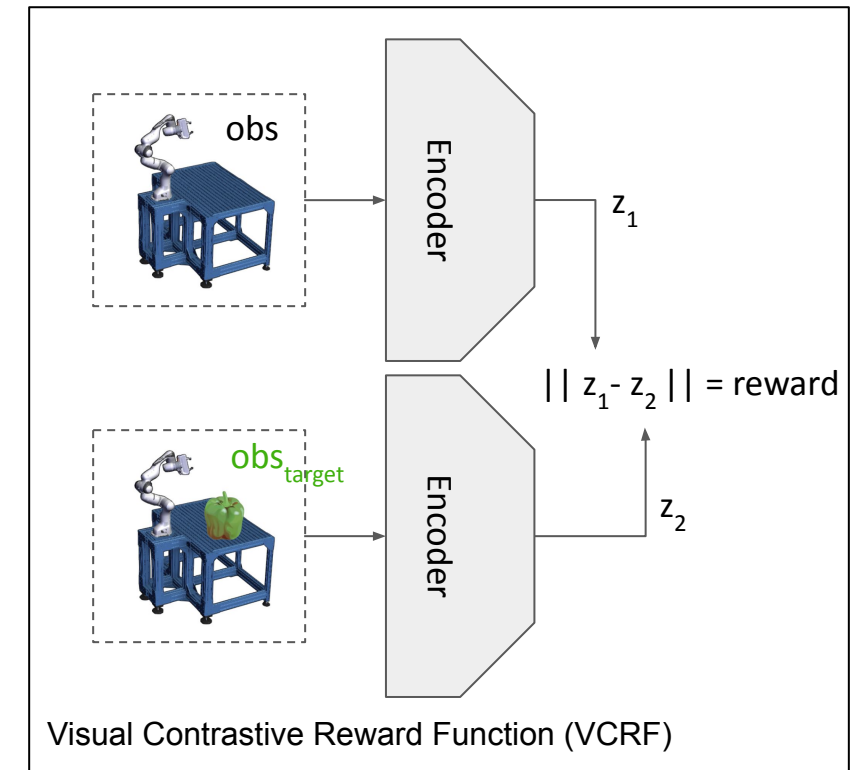
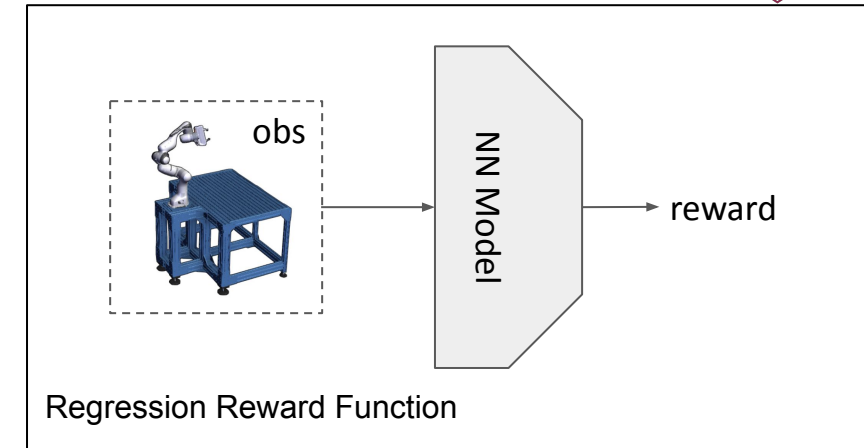


Reward Function #2: Visual Reward Function



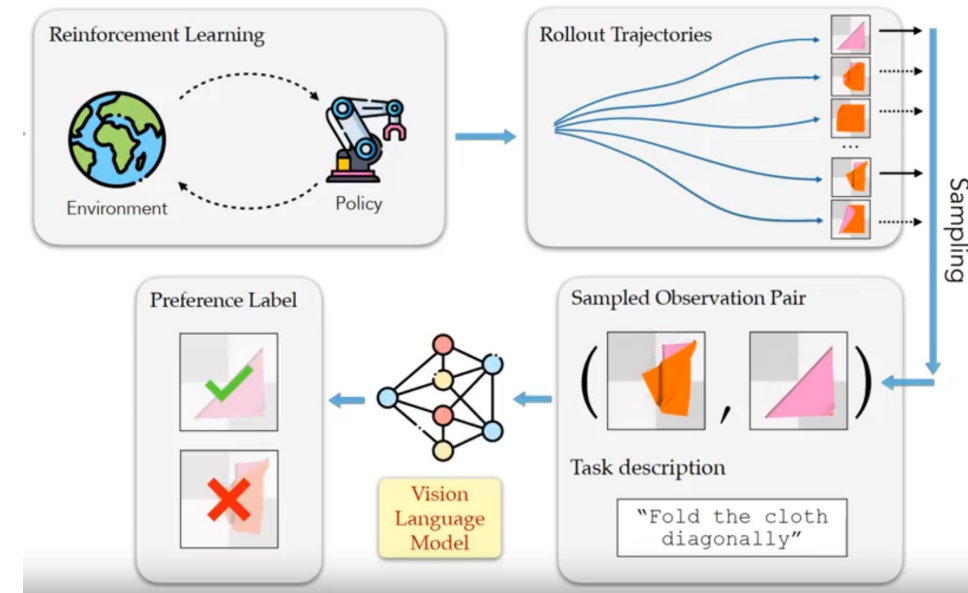
Visual reward functions map from rgb to reward. There are two common ways this is done:

1. The function has regressed on ground truth reward values
 - + Exact reward values
 - Requires reward value for every frame
2. The reward is calculated as the distance between the latent encoding of the current observation and of the goal state in the expert trajectory
 - + Able to learn through unlabeled videos - no rewards required
 - + Lots of data available (Ego4d, Youtube, ...)



Reward Function #3: VLM Grading

1. VLM used during execution
 - Inference is likely too expensive. Parallel inference is possible but 100+ sim environments with H observations each
2. VLM used during offline training. Learn a reward function from query responses
 - a. Existing work on this [0]



Section 3: Real2Sim

Yumeng He



Master

Aug 2024 – May 2026 (Expected)

University of Southern California

- Computer Science

Bachelor

Sep 2019 - May 2024 (including 1 year of CO-OP)

University of Toronto

- Honours Bachelor of Science degree with High Distinction
- Mathematics and computer science double major
- Dean's List Scholar for 4 semesters

Real2Sim - Landscape

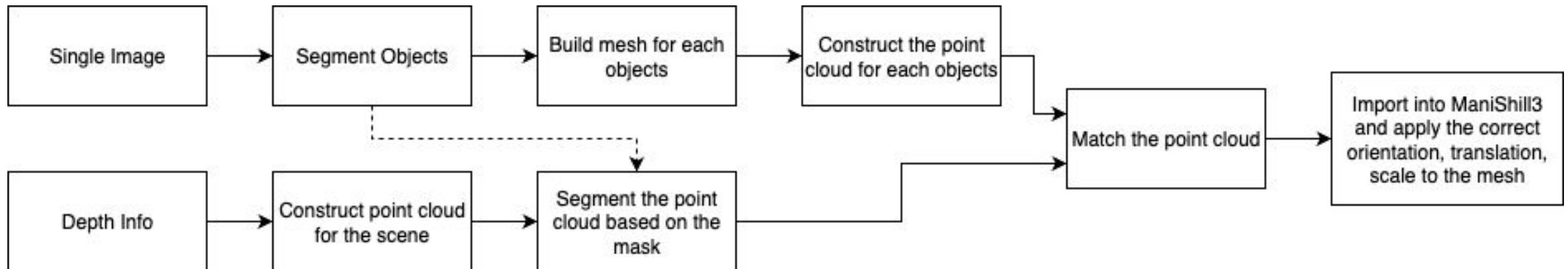


How to populate a simulator with what the robot observes?

Equivalently: How to build a digital twin?

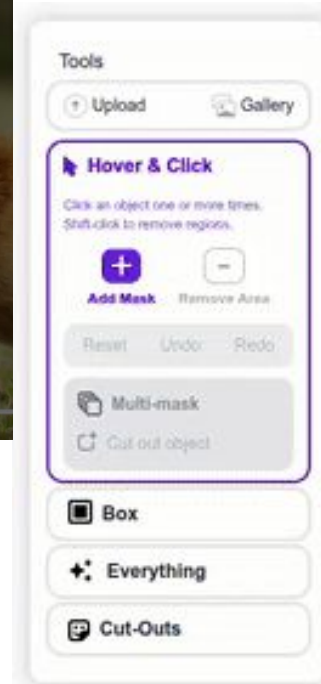
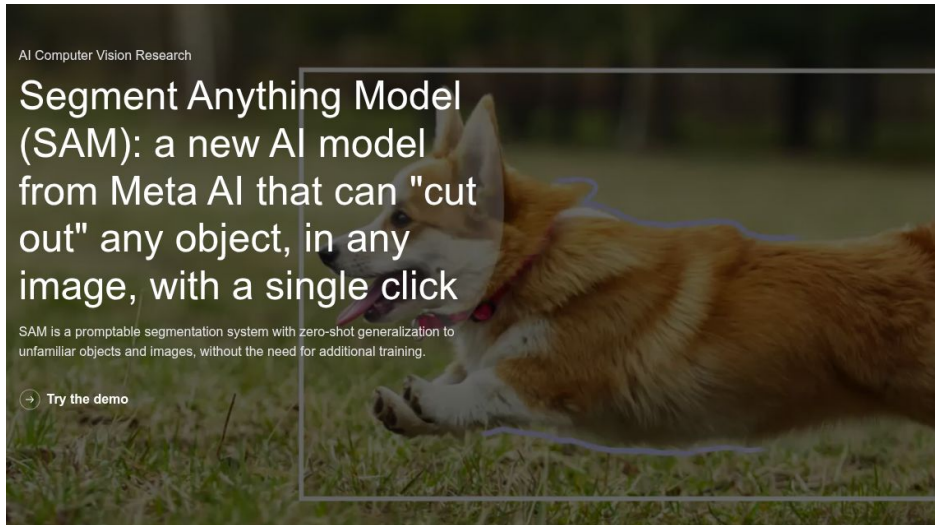
- i.e. mesh for each objects with pose (orientation, translation, scale) and simulation properties (mass, friction, etc.)

Pipeline:





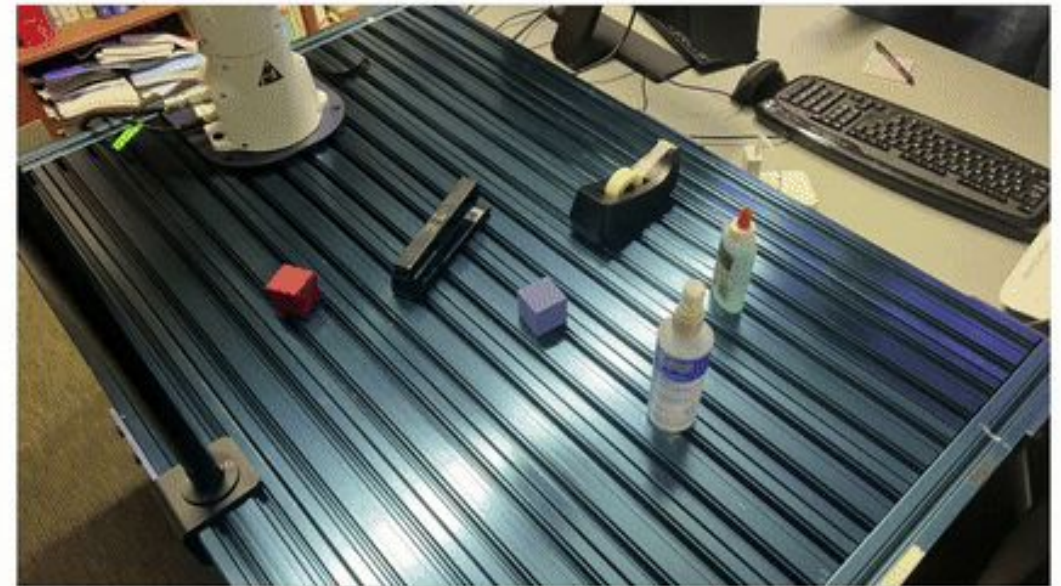
Segment 2D Images



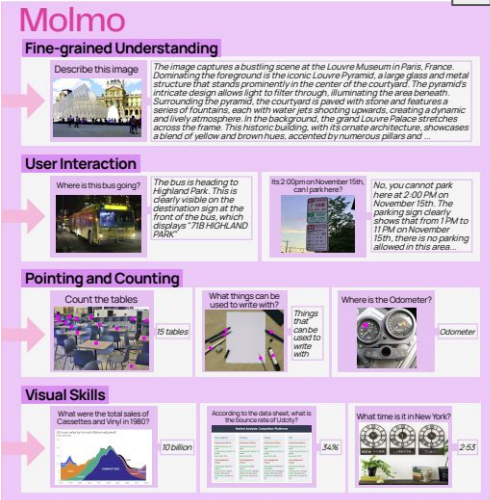
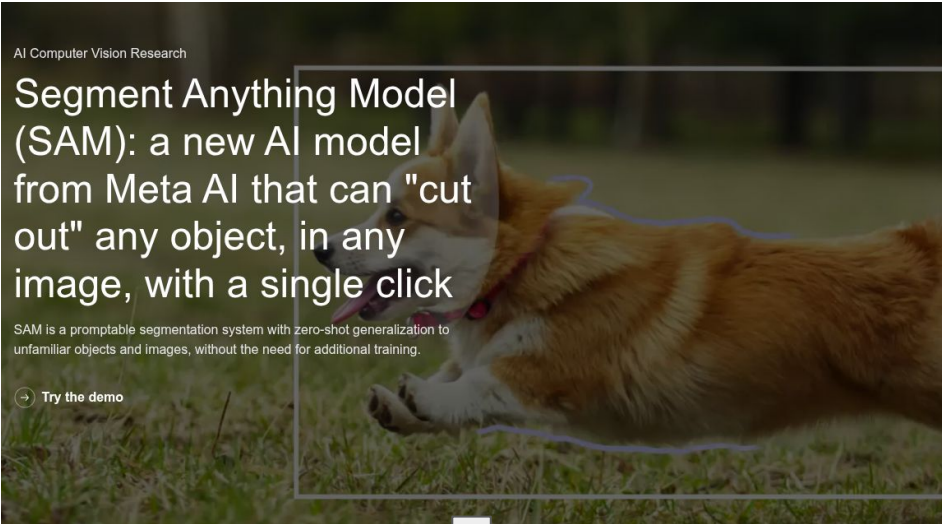
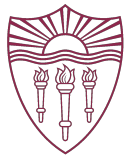
When hovering over the image, SAM is running in the browser.

Segment Anything Model (SAM)

- an interactive model to segment objects of 2D images
- segment by clicking, boxing, or automatically segment every objects



Segment 2D Images



Molmo:
select objects
based on the
text prompt



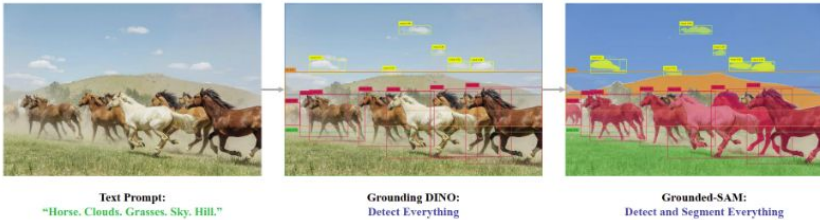
Grounded-Segment-Anything



We plan to create a very interesting demo by combining [Grounding DINO](#) and [Segment Anything](#) which aims to detect and segment anything with text inputs! And we will continue to improve it and create more interesting demos based on this foundation. And we have already released an overall technical report about our project on arXiv, please check [Grounded SAM: Assembling Open-World Models for Diverse Visual Tasks](#) for more details.

- 🔥 [Grounded SAM 2](#) is released now, which combines Grounding DINO with [SAM 2](#) for any object tracking in open-world scenarios.
- 🔥 [Grounding DINO 1.5](#) is released now, which is IDEA Research's **Most Capable** Open-World Object Detection Model!
- 🔥 [Grounding DINO](#) and [Grounded SAM](#) are now supported in Huggingface. For more convenient use, you can refer to [this documentation](#)

We are very willing to **help everyone share and promote new projects** based on Segment-Anything. Please check out here for more amazing demos and works in the community: [Highlight Extension Projects](#). You can submit a new issue (with `project` tag) or a new pull request to add new project's links.

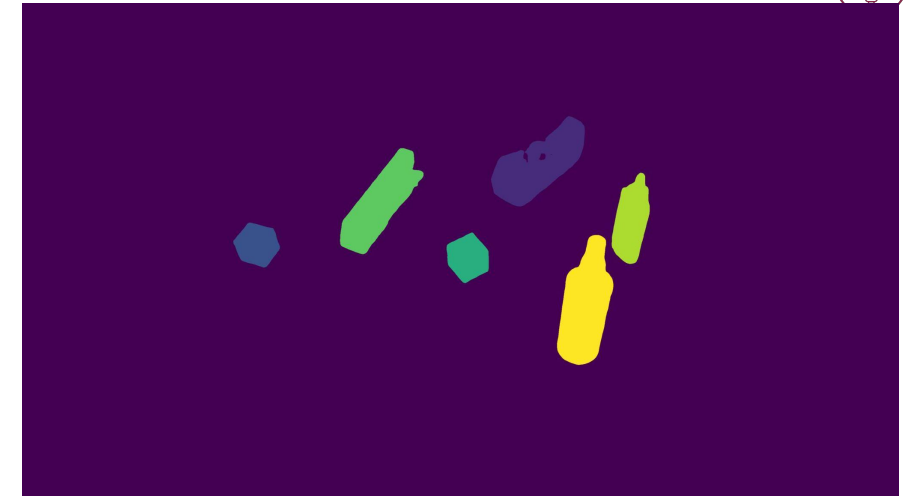
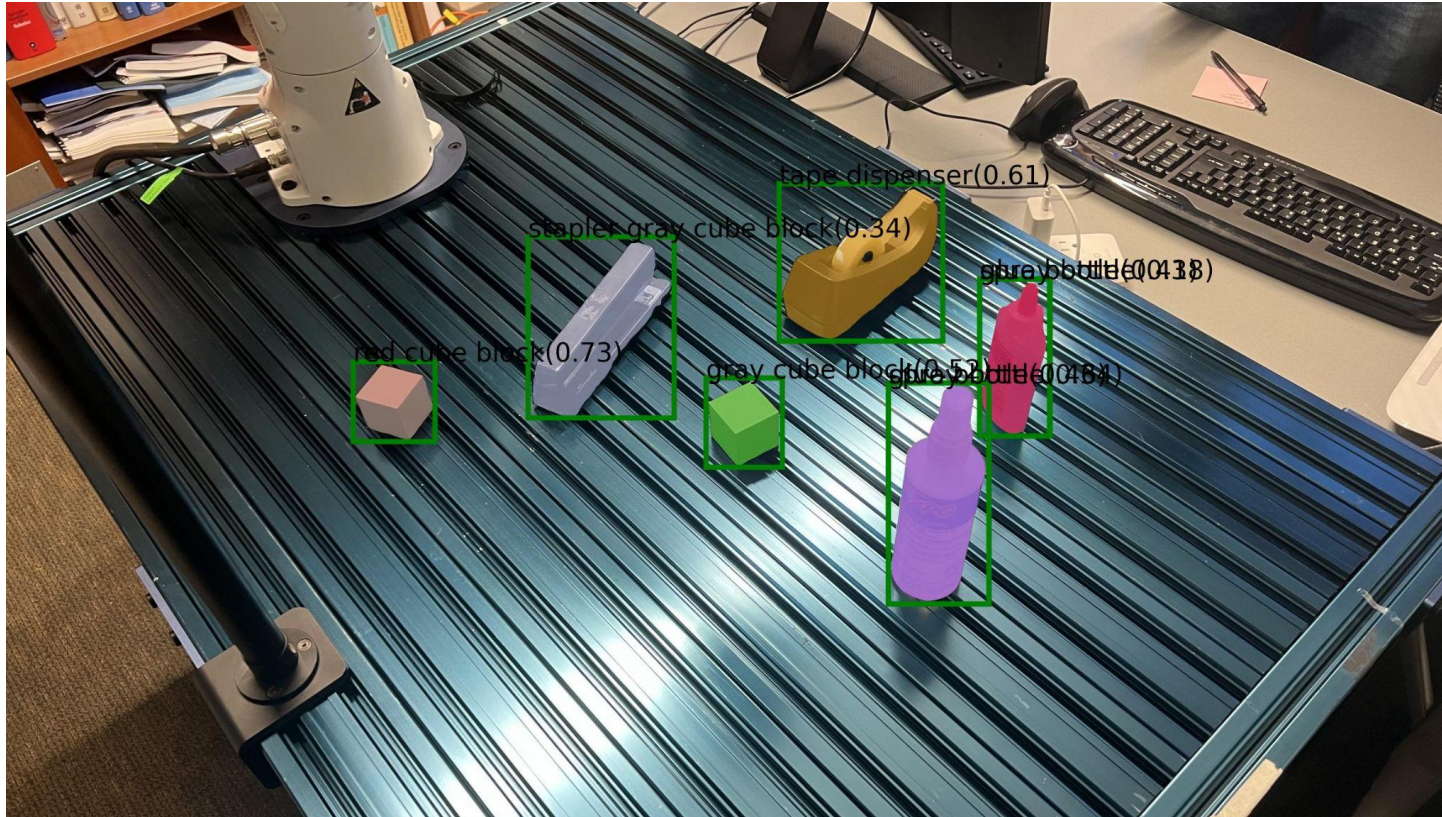


[0] Deitke, Matt, et al. "Molmo and pixmo: Open weights and open data for state-of-the-art vision-language models." Proceedings of the Computer Vision and Pattern Recognition Conference. 2025.

[1] Ren, Tianhe, et al. "Grounded sam: Assembling open-world models for diverse visual tasks." arXiv preprint arXiv:2401.14159 (2024).



Segment 2D Images



Grounded-Segment-Anything
sample results on our images



Construct 3D Mesh

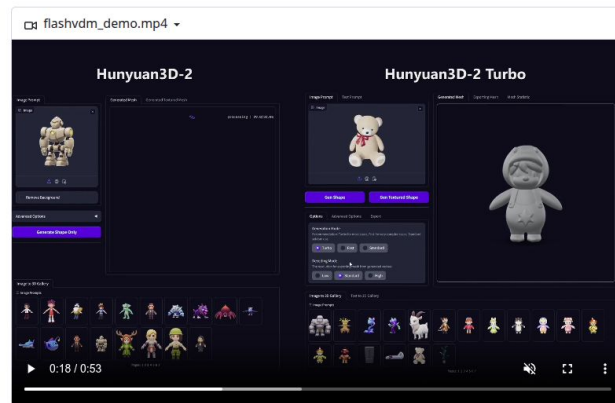


中文阅读 日本語で読む



Official Site Demo Models Page Discord Report Hunyuan Community

"Living out everyone's imagination on creating and manipulating 3D assets."



Hunyuan3D 2.0

- not fully open source
- not for free

TripoSG: High-Fidelity 3D Shape Synthesis using Large-Scale Rectified Flow Models

Arxiv 2025

Yangguang Li^{1*} Zi-xin Zou^{1*} Zexiang Liu¹ Dehu Wang¹ Yuan Liang¹ Zhipeng Yu¹
Xingchao Liu³ Yuan-Chen Guo¹ Ding Liang^{1*} Wanli Ouyang^{2,4} Yan-Pei Cao¹

¹VAST ²The University of Hong Kong ³The University of Texas at Austin ⁴Shanghai AI Laboratory

* Equal contributions; # Corresponding author

Paper Code Model Demo

Teaser



High-quality 3D shape samples from our largest TripoSG model. Covering various complex structures, diverse styles, imaginative designs, multi-object compositions, and richly detailed outputs, demonstrates its powerful generation capabilities.

TripoSG

- no texture

TRELLIS^{*}

Structured 3D Latents

for Scalable and Versatile 3D Generation

Jianfeng Xiang^{1,3} Zelong Lv^{2,3} Sicheng Xu³ Yu Deng³ Ruicheng Wang^{2,3}
Bowen Zhang^{2,3} Dong Chen³ Xin Tong³ Jialong Yang³
¹Tsinghua University ²USTC ³Microsoft Research

CVPR 2025 Highlight

Paper Arxiv Code Demo

(Note: This is the only official webpage for the TRELLIS project)



Trellis

- requires 16GB memory

[0] Zhao, Zibo, et al. "Hunyuan3d 2.0: Scaling diffusion models for high resolution textured 3d assets generation." arXiv preprint arXiv:2501.12202 (2025).

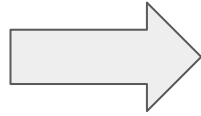
[1] Li, Yangguang, et al. "TripoSG: High-fidelity 3d shape synthesis using large-scale rectified flow models." arXiv preprint arXiv:2502.06608 (2025).

[2] Xiang, Jianfeng, et al. "Structured 3d latents for scalable and versatile 3d generation." Proceedings of the Computer Vision and Pattern Recognition Conference. 2025.



Construct 3D Mesh

Sample output of **Trellis** on our segmented object:



Next steps



Open3D:

- generate point clouds for each objects
- generate point clouds for the scene
- compare the 2 point clouds to have the correct translation, scale, and position
- import the mesh into ManiSkill3

Further enhancement:

- physical property identification
- realtime

Section 4: Project Pitch

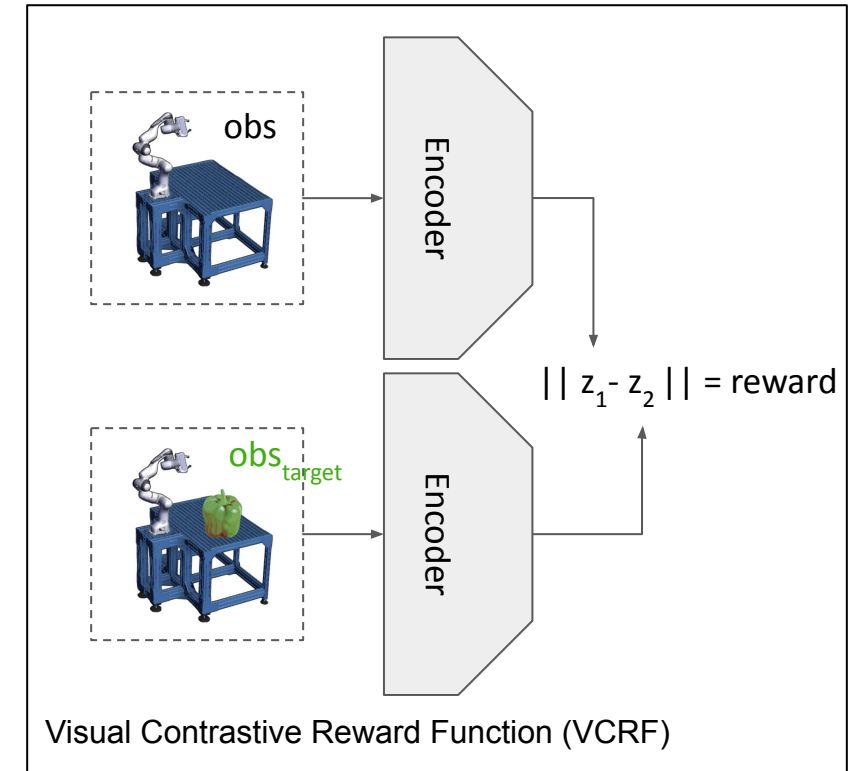


Terminology 1/4

Visual Contrastive Reward Function (VCRF): a vision based reward function. Reward is the (negative) distance between the latent encoding of the current observation and of the goal state in the expert trajectory

Pretrained Vision Model (PVM): An image encoder which generates embedding vectors for downstream control tasks. SOTA methods are typically ResNet-X or ViT models trained on egocentric videos (Ego4d, something-something, etc)

- This is the encoder in the figure on the right





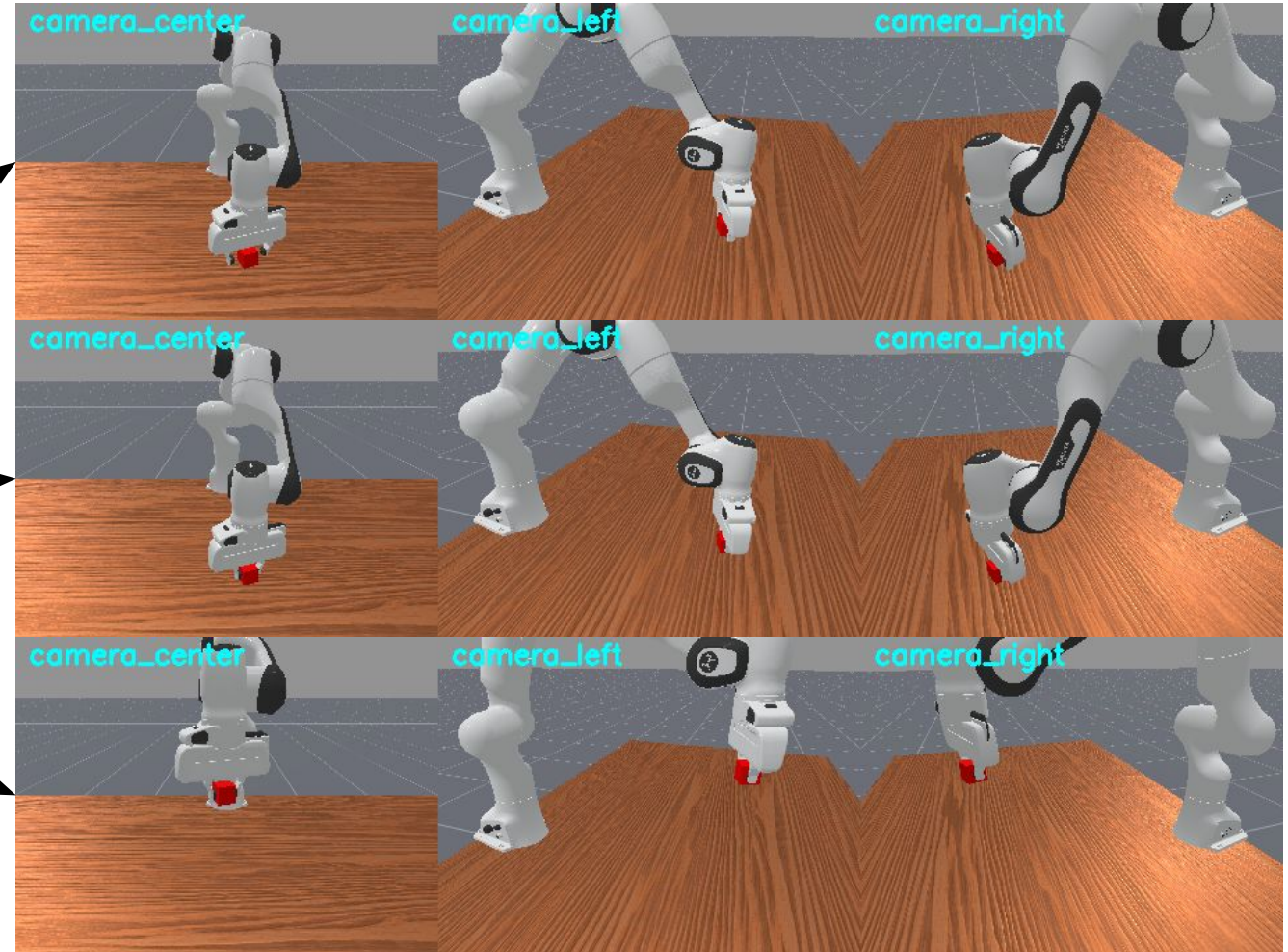
Terminology 2/4

Subgoal: The state at a time point in a demonstration that is deemed 'important', and the robot needs to reach during execution

Subgoal 0: Pre grasp pose

Subgoal 1: Post grasp pose

Subgoal 2: Place pose

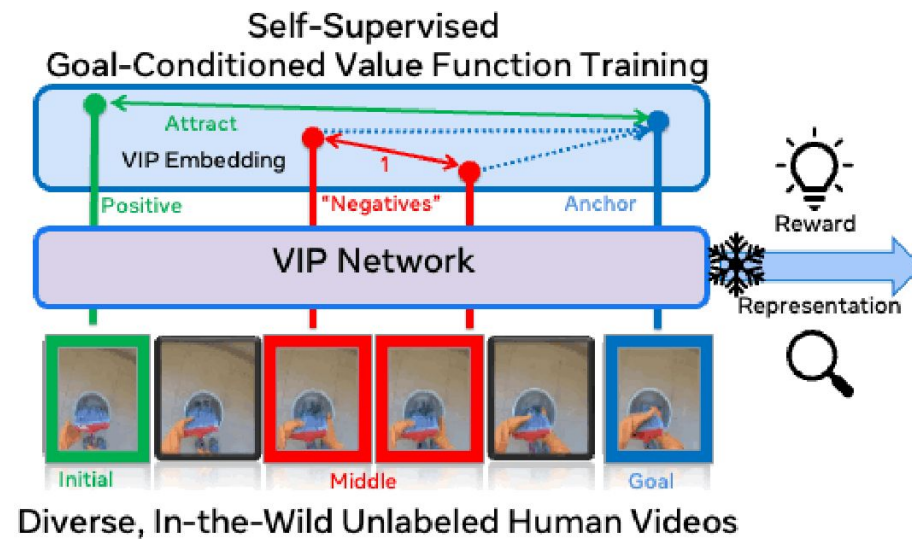




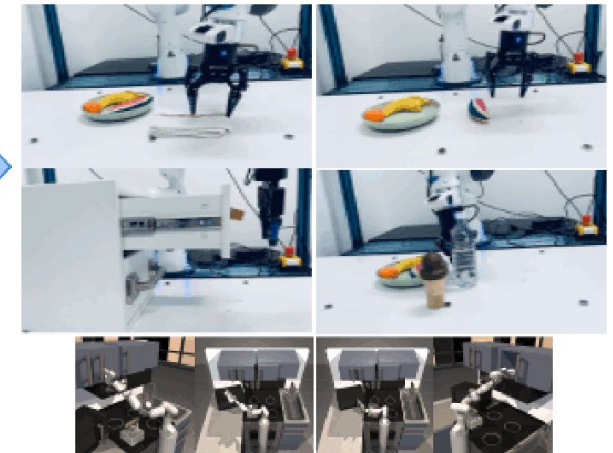
Terminology 3/4

VIP: a Pretrained Vision Model which generates embedding vectors for downstream control tasks.

VIP: Towards Universal Visual Reward and Representation Via Value-Implicit Pre-Training



Diverse Visuomotor Control:
Imitation, Trajectory Optimization, Online RL,
Few-Shot Real-World Offline RL



sites.google.com/view/vip-rl



github.com/facebookresearch/vip

Terminology 4/4



PickCube-v1: A pick in place task from ManiSkill where the robot picks a cube and brings it to a target position

[PickCube-v1-motion-planning demonstrations.mp4](#)

Proposed System: Subgoal Extraction + Tracking

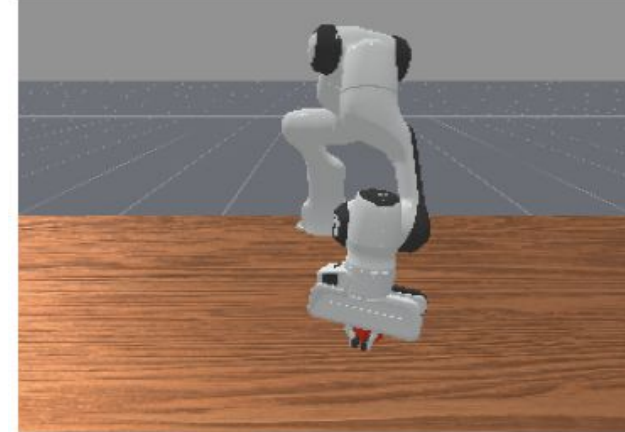


- *Subgoals* are extracted from a demonstration of a human or robot performing a task.
- The MPC controller takes actions to minimize the embedding distance between the current observation and the current subgoal
- The next subgoal is tracked once the current one is reached

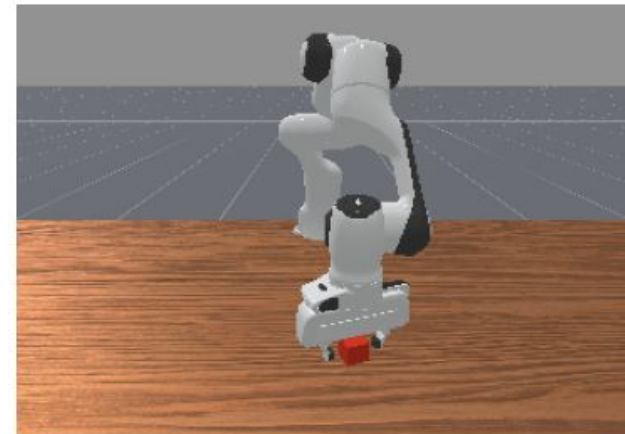
Key Components

- Subgoal partitioning: How to create subgoals? Currently using gripper starts moving, stops moving, EE starts moving, stops moving
- Subgoal transition prediction: During execution, how to predict the current subgoal is reached?
- Visual Contrastive Reward Function: Which to use?

Best Match
Embedding distance: -5.863, Env-idx: 22



Target Subgoal



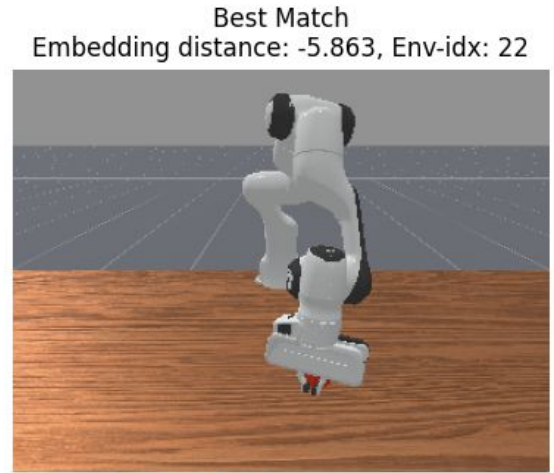
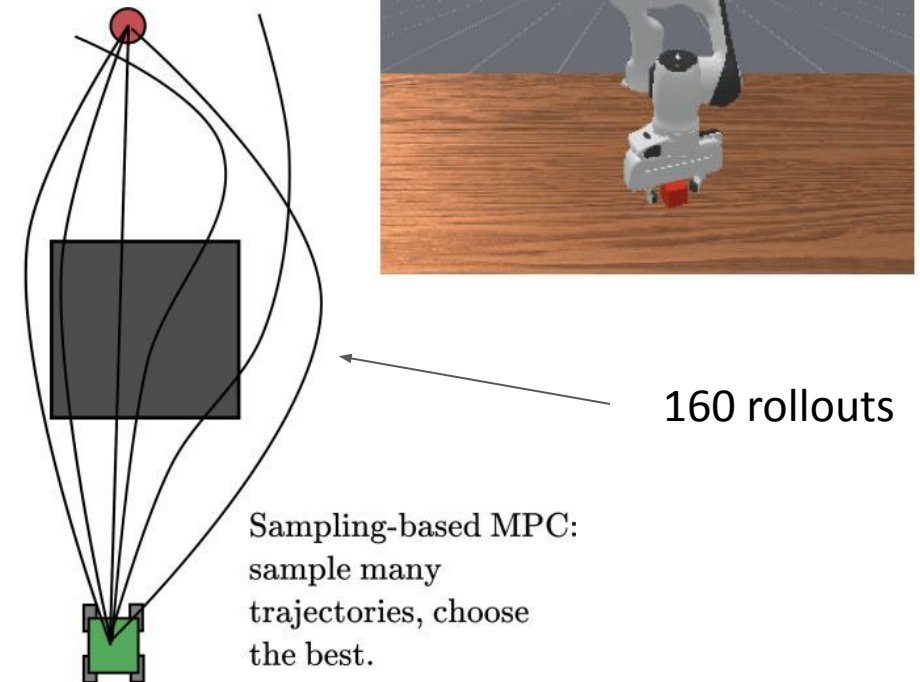
Motivating Result

With the following setup:

- 160 random end effector velocity commands -> 160 sim 'world model' environments
- 8 step horizon
- PickCube ManiSkill task
- Using VIP as a single-view VCRF
- Subgoals from motion planning solution demonstrations. Gripper start/stop motion determines subgoals

Unable to solve with single-view VCRF

Able to solve with 3 views



Project Directions



Here's where things get tricky. What's the actual contribution?

1. Develop an improved PVM. Pitch and publish it standalone from this MPC idea, *or* show that it enables few-shot manipulation with the MPC setup described before when used as a VCRF
2. A contribution on how to do subgoal extraction and transitioning. Show how it's used by the MPC controller. Use an off-the-shelf VCRF

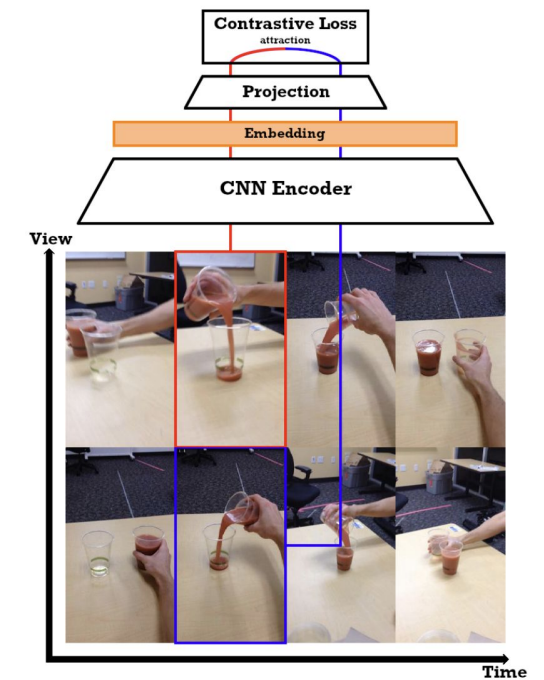
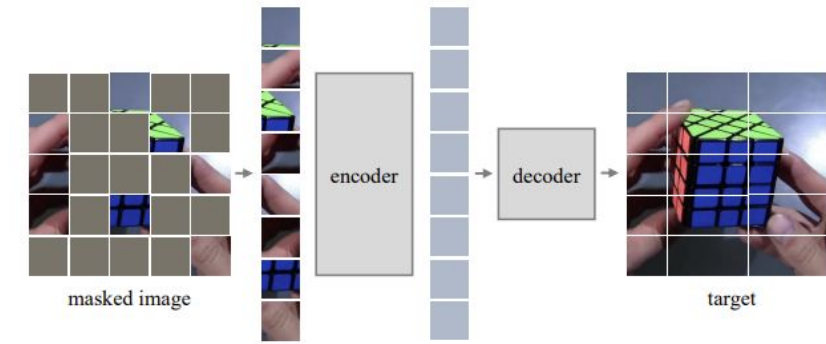
Game plan: Develop a minimum working system. Then try on increasingly difficult tasks. What stops working? What are the fundamental issues?

1. Is subgoal extraction extraction hard?
2. Subgoal transition prediction?
3. Should the two be combined?

Project Direction 1: PVM

Design choices

1. Single-view (SV) vs. Multi-view (MV)
2. Include depth?
3. Loss: Contrastive vs. masking vs. future prediction
4. Loss: temporal vs. non temporal
5. Data source



Project Direction 1: Improved PVMs

	Single view	Multi-view
RGB only	• MVP - Masked Visual Pre-training for Motor Control • Learning Visual Robotic Control Efficiently with Contrastive Pre-training and Data Augmentation • PVDR - Pre-trained Visual Dynamics Representations for Efficient Policy Learning • Reinforcement learning with action-free pre-training from videos • Real-world robot learning with masked visual pre-training • Masked World Models for Visual Control • Pri3D: Can 3D Priors Help 2D Representation Learning? • Visual Pretraining via Contrastive Predictive Model for Pixel-Based Reinforcement Learning • Exploring Visual Pre-training for Robot Manipulation: Datasets, Models and Methods • R3M: A Universal Visual Representation for Robot Manipulation	MULTI-VIEW CONTRASTIVE LEARNING FROM DEMONSTRATIONS - Data: Self collected, RLBench Multi-View Masked World Models for Visual Robotic Manipulation - Data: Self collected, RLBench Look Closer: Bridging Egocentric and Third-Person Views with Transformers for Robotic Manipulation - Data: Self collected Time-Contrastive Networks: Self-Supervised Learning from Video - Data: Self collected <u>No existing work with large scale datasets</u>
RGB + Depth	DPR - Visual Robotic Manipulation with Depth-Aware Pretraining - ScanNet dataset (static environments / not egocentric) - Depth only used during training <u>No existing work with depth during inference</u>	<u>No existing work</u>

Project Direction 1: Improved PVMs

Missing in the literature

- 1. Single-view to multi-view converter for egocentric videos. This would enable training and using multi-view VCRFs which should be better than single-view
- 2. Inclusion of depth information in PVM. Can use DepthAnything or a similar depth estimation model to convert RGB egocentric

	Single view	Multi-view
RGB only	...	... - All data is self collected or from RLBench <u>No existing work with large scale datasets</u>
RGB + Depth	... - Data isn't egocentric - Depth only used during training <u>No existing work with depth during inference</u>	<u>No existing work</u>

Project Direction 1: VCRF, Single-view -> Multi-view

Motivation:

- Multiview improves the representation ability of the PVM (this is verified for eye-in-hand)
- There are many more SV videos (youtube) than MV videos, but MV does better. Let's get the best of both worlds by translating any SV video to MV video

Idea: Create a single-view (SV) to multi-view (MV) converter for egocentric videos. Then pretrain a PVM using multiple viewpoints.

How to do novel viewpoint rendering?

1. Train a gaussian splat (or nerf)
2. Project from a point cloud. Get the depth channel using a depth prediction model
3. Build a SLAM map

Questions to answer

1. For RL: what is the impact on success rate and sample efficiency?
2. For RL, IL: What is the impact on visual robustness to out of distribution observations?
- ...
3. Can we solve manipulation tasks few shot now using MPC + subgoal tracking?

Project Direction 1: VCRF, RGB -> RGBD

Motivation:

- Depth information improves the representation ability of the VCFG. (need to verify this)
- There are many more SV videos (youtube) than MV videos, but MV does better. Let's get the best of both worlds by translating any SV video to MV video

Idea: Convert RGB egocentric videos to RGBD using DepthAnything or a similar depth estimation model

Demonstrate RGBD does better on downstream control tasks.

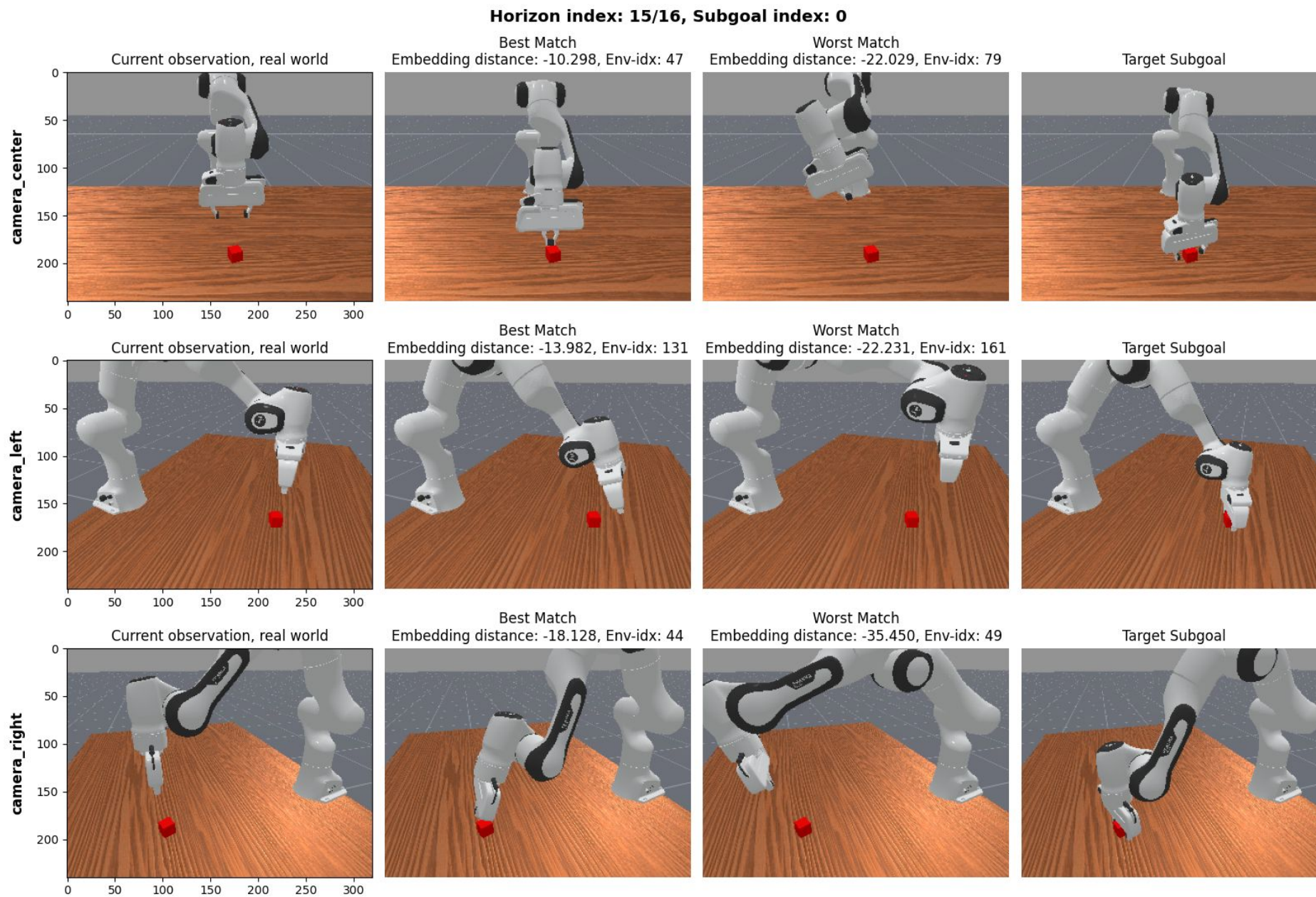
(same) Questions to answer

1. For RL: what is the impact on success rate and sample efficiency?
2. For RL, IL: What is the impact on visual robustness to out of distribution observations?

...

3. Can we solve manipulation tasks few shot now using MPC + subgoal tracking?

Project Direction 2: Subgoal extraction + tracking



Project Direction 2: Subgoal extraction + tracking

Subgoal extraction

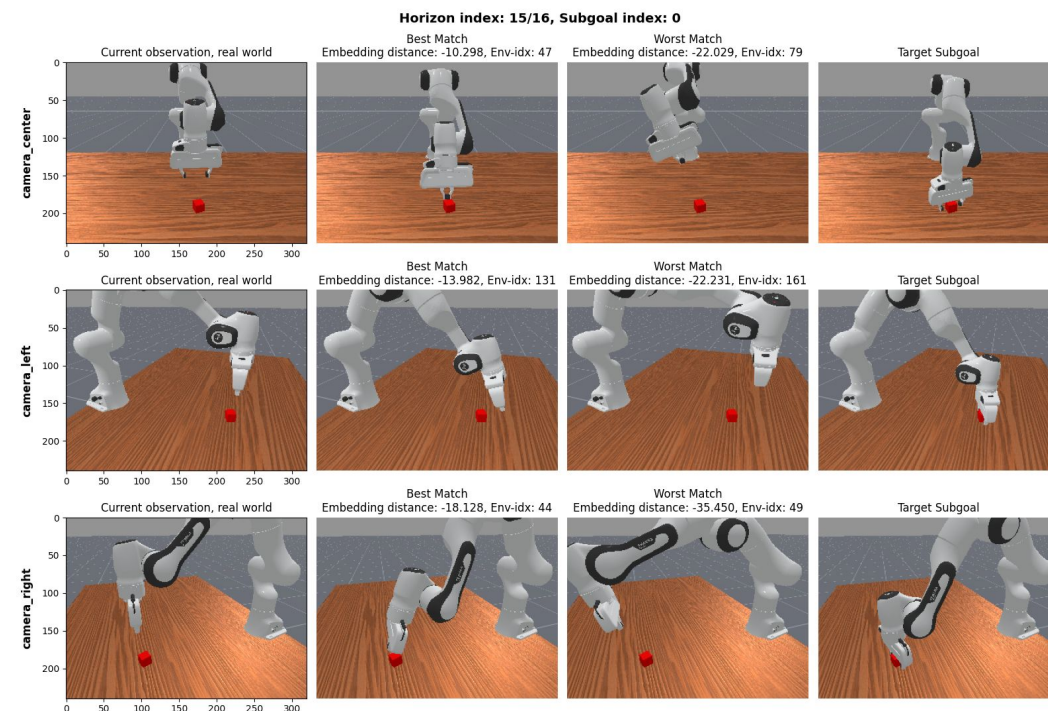
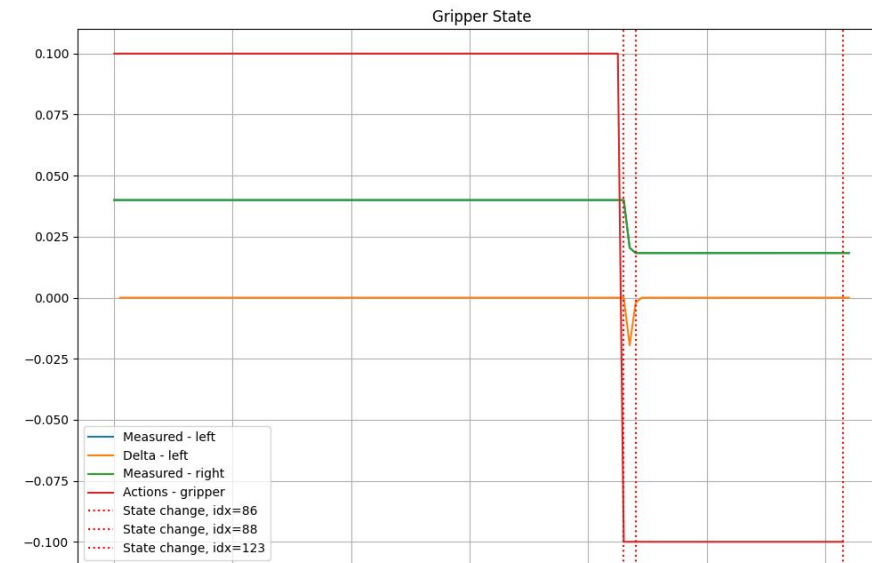
How to generate subgoals?

- Currently: when gripper starts or stops moving.
- Other options:
 - 1: VIP Universal Decomposer code
 - 2: Look at velocity of end effector / joint configuration
 - 3: VLM to create subgoals.

Subgoal tracking

During execution, when have we reached the current goal?

- Currently: a single threshold value is used for all cameras.
- Need the max embedding distance to be below handset threshold.
- Other options:
 - VLM to generate per-camera thresholds, or per-camera, per-subgoal thresholds
 - MLP to detect when successful
 - VLM to detect when successful



Project Direction 2: Subgoal extraction + tracking

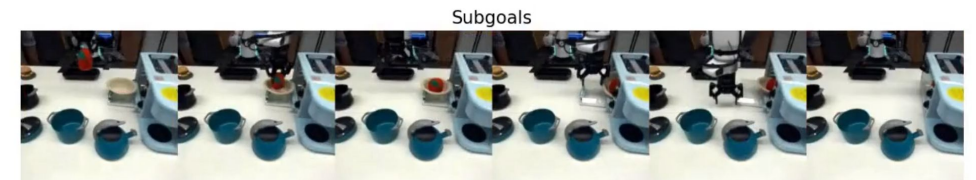
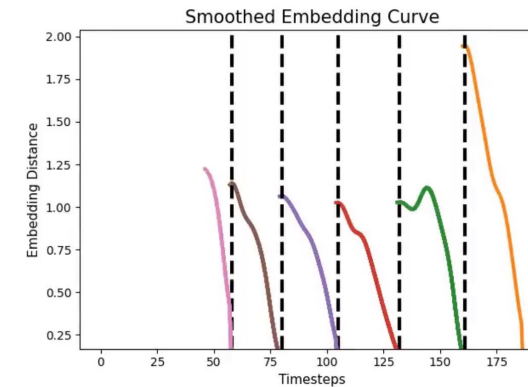
Subgoal extraction

How to generate subgoals?

- Currently: when gripper starts or stops moving.
- Other options:
 - 1: VIP Universal Decomposer code
 - 2: Look at velocity of end effector / joint configuration
 - 3: VLM to create subgoals.

Universal Visual Decomposer: Long-Horizon Manipulation Made Easy

- <https://zcczhang.github.io/UVD/>



Fin

Leading Idea #1: RL to generate reward function



RL outputs the weights for a set of feature functions. MPC controller optimizes the weighted sum of features during evaluation

Features:

- `distance(obj_a, obj_b)`
- `velocity(obj)`
- `relative-distance-x/y/z/roll/pitch/yaw(obj_a, obj_b)`

`obj` can be a link, object, etc.

Current approach: Subgoal Extraction + Tracking



From a demonstration of a human performing a task, generate observations at the terminal states of each subtask. MPC Controller takes actions to reach the end of each subtask using the visual difference between the subtask termination image and the current image

- VIP model can be used as visual difference reward function
- Subgoals are partitioned according to changes in contact/no-contact state with the world
 - Hand contact state estimated using model from [0]
- Need visual difference score between current state, target state to monotonically decrease